Containers, Modern DevOps, and Migration Strategy

CSC-535 Cloud DevOps

Muhammad Tausif

Department of Computer Science
QAU

Lecture Roadmap

1. Containers and Modern DevOps
2. Modern DevOps vs Traditional DevOps
3. Migration from Virtual Machines to Containers
4. Structured Migration Strategy
5. What Should Go Into Containers?
6. Breaking Applications into Smaller Pieces
7. Reality of Virtual Machines in the Container Era
8. Advanced Operational Concepts
9. Economic and Organizational Impact
10. Final Integration
11. Silos in DevOps and Microservices
12. 1. Organizational Silos
13. 2. Data Silos
14. 3. Knowledge Silos
15. Why Silos Kill Innovation
16. Hypervisor and Virtual Machine Monitor (VMM)
17. How a Hypervisor Works

Containers as the Core of Modern DevOps

- Containers standardize application packaging.
- They bundle:
  - Application code
  - Runtime (e.g., Node, Python, JVM)
  - System libraries
  - Configuration dependencies
- Enable **environment consistency** across Dev, Test, Production.
- Foundational technology behind:
  - Cloud-native systems
  - Microservices architecture
  - CI/CD automation
- Reduce friction between development and operations teams.

Why Containers Fit DevOps Philosophy

DevOps emphasizes:

- Collaboration between Dev and Ops.
- Automation over manual processes.
- Continuous delivery of value.

Containers enable:

- Infrastructure as Code.
- Immutable deployments.
- Reproducible builds.
- Faster feedback cycles.

Result: Reduced deployment risk and faster innovation.

Typical Container-Based Development Workflow

1. Write application code.
2. Define environment using Dockerfile.
3. Build container image.
4. Run and test locally.
5. Push image to registry.
6. Deploy using orchestrator.

Each step is:

- Repeatable
- Automated
- Version-controlled

Dockerfile as Configuration-as-Code

A Dockerfile:

- Starts from a base image.
- Installs required dependencies.
- Copies application code.
- Defines working directory.
- Sets environment variables.
- Specifies entrypoint/command.

Key Concept:

$$Environment = Code$$

No hidden configuration on servers.

Containers in Continuous Integration (CI)

CI Pipeline Stages:

- Code pushed to Git repository.
- Automated build triggered.
- Container image created.
- Unit tests executed inside container.
- Static code analysis.
- Security vulnerability scanning.

Containers ensure:

- Same runtime during testing as production.

Containers in Continuous Deployment (CD)

CD Pipeline Stages:

- Image pulled from registry.
- Deployment to staging cluster.
- Smoke and integration testing.
- Approval gates (optional).
- Automated production rollout.
- Monitoring and rollback capability.

Immutable Deployment Model: Deploy new container version instead of modifying existing one.

Ephemeral Infrastructure Concept

Containers are:

- Short-lived.
- Replaceable.
- Stateless (ideally).

If container fails:

- Destroy it.
- Launch a new instance.

This contrasts with:

- Traditional servers (repair mindset).

Shift: Repair → Replace

Containers and Orchestration

In production, containers are managed by orchestrators:

- Kubernetes
- Docker Swarm

Orchestrator responsibilities:

- Scheduling containers on nodes.
- Auto-scaling based on load.
- Restarting failed containers.
- Service discovery.
- Rolling updates.

Traditional DevOps Model

Characteristics:

- VM-based deployment.
- Manual server configuration.
- Separate Dev, QA, Ops roles.
- Tool-centric automation (Jenkins, scripts).
- Infrastructure maintained manually.

Challenges:

- Configuration drift.
- Slow scaling.
- Heavy operational overhead.

Modern DevOps Model

Characteristics:

- Infrastructure as Code (IaC).
- Containerized workloads.
- Automated CI/CD pipelines.
- Cloud-native infrastructure.
- Platform-driven operations.

Focus shifts from:

$$\textit{Managing Servers} \rightarrow \textit{Managing Systems as Code}$$

Shift Toward Cloud-Native Architectures

Cloud-native principles:

- Microservices.
- Containers.
- Declarative APIs.
- Elastic scaling.
- Resilience by design.

Containers act as:

- The runtime abstraction layer.
- The packaging format for cloud workloads.

Why Migrate from VMs to Containers?

Business Drivers:

- Faster release cycles.
- Reduced infrastructure cost.
- Improved scalability.
- Higher resource efficiency.
- Simplified automation.

Technical Drivers:

- Lightweight isolation.
- Better portability.
- Simplified dependency management.

Migration is Complex and Strategic

Migration is not just technical:

- Legacy systems tightly coupled.
- Hidden dependencies.
- Organizational resistance.
- Operational mindset shift required.

Migration requires:

- Structured assessment.
- Risk mitigation.
- Phased execution.
- Continuous validation.

High-Level Migration Phases

A structured migration typically follows these phases:

1. Discovery
2. Application Requirement Assessment
3. Container Infrastructure Design
4. Containerization of the Application
5. Testing (Functional + Non-Functional)
6. Deployment and Rollout

Important:

- Migration is iterative, not strictly linear.
- Feedback from later stages may require revisiting earlier stages.

Phase 1: Discovery

The discovery phase focuses on understanding the current system.
Key questions:

- What are the main components of the application?
- What services does it depend on?
- What external systems are integrated?
- Where is state stored?
- What configuration exists outside the application?

Goal:

- Build a complete inventory of the application ecosystem.

Discovery: Dependency Mapping

You must identify:

- OS-level dependencies (system packages).
- Runtime dependencies (language versions).
- Network dependencies (external APIs, databases).
- File system dependencies (local storage paths).
- Scheduled tasks (cron jobs, background services).

Hidden dependencies are the biggest risk during migration.

Phase 2: Application Requirement Assessment

After discovery, evaluate feasibility.
Assess:

- Is the application stateless or stateful?
- Can it be split into smaller components?
- Are there strict performance constraints?
- Are there regulatory or compliance constraints?
- What is acceptable downtime?

Outcome:

- Decide whether to migrate fully, partially, or not at all.

Stateless vs Stateful Workloads

Stateless Applications:

- Do not store session data locally.
- Easy to replicate and scale.
- Ideal candidates for containerization.

Stateful Applications:

- Maintain persistent storage.
- Examples: Databases, message queues.
- Require persistent volumes and careful design.

Stateless workloads should usually be migrated first.

Risk and Mitigation Strategy

During assessment, define:

- Rollback strategy.
- Data backup procedures.
- High availability requirements.
- Monitoring requirements.
- Security exposure changes.

Migration without rollback planning is operationally dangerous.

Phase 3: Container Infrastructure Design

Now design where and how containers will run.
Decisions include:

- On-premise vs Cloud.
- Managed Kubernetes vs Self-managed.
- Single cluster or multiple clusters.
- Network model (bridge, overlay, etc.).
- Storage backend selection.

Infrastructure decisions affect scalability and cost.

Capacity and Scaling Planning

Estimate:

- Expected traffic load.
- Peak vs average usage.
- Number of replicas required.
- Resource requirements (CPU, Memory).
- Auto-scaling thresholds.

Container platforms enable horizontal scaling:

$$Scale\ Out > Scale\ Up$$

Operational Readiness Planning

Before containerization, ensure:

- Logging aggregation system exists.
- Monitoring system (metrics + alerts).
- Image registry access control.
- CI/CD pipeline integration.
- Secret management solution.

Operations must be prepared before deployment.

Phase 4: Containerizing the Application

This is the technical execution phase.
Steps:

- Write Dockerfile.
- Reproduce installation steps.
- Remove VM-specific configurations.
- Externalize configuration.
- Test locally inside container.

Goal:

- Achieve runtime parity with VM behavior.

Re-Architecting During Migration

Migration may expose architectural issues:

- Tight coupling between modules.
- Hardcoded configuration paths.
- Local file system assumptions.
- Manual startup scripts.

Opportunity:

- Refactor toward microservices.
- Introduce APIs.
- Separate business logic from infrastructure logic.

Phase 5: Functional Testing

Verify correctness:

- Run regression test suite.
- Compare VM vs Container outputs.
- Validate database interactions.
- Validate external integrations.

Testing ensures:

- No behavioral deviation introduced.

Phase 5: Non-Functional Testing

Evaluate performance and resilience:

- Load testing.
- Stress testing.
- Resource profiling.
- Startup latency measurement.
- Security penetration testing.

Containers may introduce:

- Different I/O characteristics.
- Networking differences.

Phase 6: Deployment Strategy

Deployment approaches:

- Rolling Update
- Blue-Green Deployment
- Canary Deployment
- Shadow Deployment

Goal:

- Minimize downtime.
- Reduce user impact.
- Allow safe rollback.

Continuous Feedback Loop

Migration does not end at deployment.
Post-deployment activities:

- Monitor metrics.
- Collect logs.
- Analyze performance trends.
- Improve container configuration.
- Iterate on architecture.

Modern DevOps is a continuous improvement cycle.

Evaluating Workloads for Containerization

Not every application should be containerized immediately.
Key evaluation factors:

- Stateless vs Stateful behavior.
- Dependency complexity.
- Operational ownership.
- Performance sensitivity.
- Regulatory constraints.

Containerization must align with business and operational maturity.

Ideal Candidates for Containers

Best suited workloads:

- REST APIs
- Web frontends
- Background workers
- Microservices
- Batch processing jobs

Characteristics:

- Minimal local state.
- Clear dependency boundaries.
- Horizontally scalable.

Stateful Applications in Containers

Examples:

- Databases
- Message brokers
- Caching systems

Challenges:

- Persistent storage management.
- Backup strategies.
- Data integrity.
- Performance consistency.

Requires persistent volumes and careful orchestration design.

Should Databases Be Containerized?

Technically possible, but context matters.
Consider:

- Production-grade storage backend.
- Dedicated DBA management.
- High availability configuration.
- Replication strategies.

In early migration:

- Often better to externalize database first.

Gradual Migration Strategy

Safer migration approach:

1. Containerize stateless services first.
2. Keep database external.
3. Introduce persistent volumes later.
4. Move stateful services once platform stabilizes.

This reduces operational risk.

Monolith vs Microservices

Monolith:

- Single deployable unit.
- Shared codebase.
- Scales as one unit.

Microservices:

- Independently deployable services.
- Independent scaling.
- Loosely coupled.

Containers naturally support microservices architecture.

Benefits of Service Decomposition

Advantages:

- Faster independent releases.
- Fault isolation.
- Better scalability.
- Smaller codebases per service.
- Clear ownership boundaries.

Risks of Over-Decomposition

Too many services can cause:

- Network latency overhead.
- Complex service discovery.
- Debugging difficulty.
- Increased operational burden.
- Distributed transaction complexity.

Microservices require mature observability.

Service Boundary Design Principles

When splitting services:

- Align with business domains.
- Minimize shared databases.
- Use APIs for communication.
- Prefer asynchronous messaging when possible.
- Avoid tight coupling between services.

Domain-driven design is often helpful.

Are Virtual Machines Obsolete?

No.

Reasons:

- Legacy applications.
- Specialized workloads.
- Stronger isolation requirements.
- Cloud infrastructure itself runs on VMs.

Containers often run inside virtual machines.

Containers on Virtual Machines

In cloud environments:

- VM provides hardware abstraction.
- Containers provide application abstraction.
- Orchestrator manages container lifecycle.

Layered abstraction model:

$$Hardware \rightarrow VM \rightarrow Container \rightarrow Application$$

Observability in Container Systems

Modern observability includes:

- Metrics (CPU, Memory, Latency)
- Logs (centralized logging)
- Traces (distributed tracing)
- Alerts and dashboards

Without observability, microservices become opaque.

Service Mesh Concept

A service mesh provides:

- Secure service-to-service communication.
- Traffic routing.
- Mutual TLS encryption.
- Observability at network layer.
- Fine-grained traffic control.

Common tools:

- Istio
- Linkerd

Security in Containerized Environments

Security considerations:

- Image vulnerability scanning.
- Least-privilege containers.
- Secret management.
- Network policy enforcement.
- Runtime security monitoring.

Security must be integrated into CI/CD.

DevSecOps Integration

DevSecOps means:

- Security scanning during build.
- Dependency vulnerability checks.
- Container image scanning.
- Infrastructure as Code validation.

Security shifts left in development lifecycle.

Economic Benefits of Containers

Containers enable:

- Higher resource density.
- Reduced infrastructure waste.
- Faster release cycles.
- Reduced downtime costs.
- Improved team productivity.

Organizational Transformation

Containers change:

- Team responsibilities.
- Deployment ownership.
- Release management processes.
- Infrastructure management models.

Requires cultural transformation.

End-to-End Container Lifecycle

Complete lifecycle:

1. Code
2. Build
3. Test
4. Package
5. Deploy
6. Monitor
7. Improve

Continuous cycle of improvement.

Key Strategic Takeaways

- Containers solve environment inconsistency.
- Migration must be structured and phased.
- Stateless workloads first.
- Orchestration is essential at scale.
- Observability and security are mandatory.
- VMs and containers coexist.

Conclusion

We have covered:

- Containers and modern DevOps.
- CI/CD integration.
- Migration from VMs.
- Infrastructure planning.
- Testing and rollout.
- Architectural decomposition.
- Operational and security considerations.

Containers form the foundation of cloud-native computing and modern DevOps.

What Are Silos?

In a DevOps and Microservices context, **silos** refer to:

- Isolated pockets within an organization.
- Teams operating independently.
- Data confined within boundaries.
- Lack of communication and shared goals.

Silos represent:

- The opposite of collaboration.
- The anti-pattern to DevOps principles.
- A barrier to agility and innovation.

Why Silos Are an Anti-Pattern

Everything DevOps promotes:

- Collaboration
- Automation
- Shared ownership
- Fast feedback
- Continuous improvement

Silos cause:

- Isolation
- Delays
- Blame culture
- Slow delivery
- Reduced transparency

Therefore:

$$\textit{Silos} \rightarrow \textit{Opposite of DevOps}$$

Organizational Silos: The Wall of Confusion

Organizational silos occur when:

- Development
- Operations
- QA

operate as separate kingdoms.
This creates:

- Poor communication.
- Conflicting goals.
- Lack of shared accountability.

This is often called: **"The Wall of Confusion"**

The Symptom: Throwing Code Over the Wall

Traditional workflow:

1. Developers write code.
2. Developers complete their work.
3. Code is handed to Operations.
4. Operations deploy and manage it.

This is known as: **"Throwing it over the wall"**
Developers are no longer responsible once it is handed off.

The Conflict of Incentives

Different incentives create tension:

Developers are incentivized to:

- Ship new features.
- Deliver change quickly.
- Move fast.

Operations is incentivized to:

- Maintain stability.
- Ensure uptime.
- Reduce incidents.

Without shared responsibility:

- Developers blame Ops for slow deployment.
- Ops blames Developers for unstable releases.

The DevOps Fix: Breaking Organizational Silos

DevOps resolves this by:

- Creating cross-functional teams.
- Sharing responsibility from code to cloud.
- Encouraging collaboration between Dev, QA, and Ops.
- Aligning incentives toward product success.

Key principle:

$$\textit{You build it} \Rightarrow \textit{You run it}$$

Everyone owns reliability and delivery.

Data Silos in Microservices

In microservices architectures:

- Each service often owns its own database.
- Data is isolated per service.
- Direct database sharing is discouraged.

A data silo occurs when:

- Data becomes trapped.
- Other services cannot access it efficiently.

The Data Silo Problem

Example scenario:

- The Shipping Service needs customer data.
- Customer data exists in Account Service.
- There is no API or event stream.

Result:

- Progress stalls.
- Tight coupling attempts appear.
- Teams bypass architecture rules.

Data isolation without communication leads to architectural bottlenecks.

Fixing Data Silos

Solutions include:

- Clear service APIs.
- API Gateway patterns.
- Event-Driven Architecture.
- Message brokers for data propagation.

Key principle:

$$\textit{Loose coupling} + \textit{Controlled data flow}$$

Services must communicate through defined interfaces.

Knowledge Silos

Knowledge silos occur when:

- Only one person understands a deployment script.
- Only one engineer knows legacy infrastructure.
- Documentation is missing or outdated.

This individual becomes: **The Silo**

The Risk of Knowledge Silos

Risks include:

- If the person leaves, progress stops.
- If the person is unavailable, deployment fails.
- Incident recovery slows dramatically.
- Organization becomes fragile.

Delivery pipeline dependency on one person:

Single Point of Failure

The DevOps Fix for Knowledge Silos

DevOps mitigates this using:

- Infrastructure as Code (IaC).
- Immutable deployments.
- Version-controlled configuration.
- Automated pipelines.

Benefits:

- Environment documented in code.
- Transparent infrastructure.
- Reproducibility.
- Shared ownership.

Slower Feedback

In siloed organizations:

- Information moves slowly.
- Requests require approvals.
- Teams wait for handoffs.

Instead of:

$$\textit{Seconds of automation}$$

You get:

$$\textit{Days of coordination}$$

Slow feedback reduces innovation speed.

Duplicate Effort

Without communication:

- Two teams may build the same internal tool.
- Monitoring solutions may be duplicated.
- Scripts may be recreated independently.

Consequences:

- Wasted resources.
- Increased maintenance cost.
- Reduced efficiency.

Low Morale and Ownership Loss

Silos affect people:

- Teams feel disconnected from impact.
- Individuals feel like "cogs in a machine."
- Lack of ownership reduces motivation.
- Blame culture increases stress.

DevOps encourages:

- Product ownership.
- Shared success.
- Shared responsibility.
- Continuous learning.

Final Reflection: Breaking Silos

Breaking silos enables:

- Faster delivery.
- Stronger collaboration.
- Resilient architecture.
- Transparent operations.
- Sustainable innovation.

DevOps is not only technical. It is organizational transformation.

What is a Hypervisor?

A **Hypervisor**, also known as a:

- Virtual Machine Monitor (VMM)

is a specialized layer of:

- Software
- Firmware
- Hardware

that enables virtualization by:

- Creating multiple Virtual Machines (VMs)
- Managing multiple VMs
- Running them on a single physical computer

Role of the Hypervisor

The hypervisor acts as a:
"Traffic Controller"
It abstracts:

- Physical CPU
- Physical Memory (RAM)
- Storage
- Network interfaces

from the guest operating systems.
Result:

- Multiple operating systems can share hardware.
- They do not interfere with each other.

Core Functions of a Hypervisor

A hypervisor performs three fundamental tasks:

1. Abstraction
2. Resource Allocation
3. Isolation

These three principles define virtualization behavior.

1. Abstraction

Abstraction means:

- The hypervisor creates a virtual version of physical hardware.
- Each VM sees:
  - Virtual CPU
  - Virtual RAM
  - Virtual Disk
  - Virtual Network Card
- The VM believes it is running on real hardware.

In reality:

$$\textit{Virtual Hardware} \rightarrow \textit{Mapped to Physical Hardware}$$

2. Resource Allocation

The hypervisor dynamically:

- Pools physical resources.
- Allocates CPU cores to each VM.
- Assigns RAM based on configuration.
- Distributes disk and network bandwidth.

Allocation can be:

- Static (fixed allocation)
- Dynamic (based on workload demand)

Goal: Efficient utilization of physical hardware.

3. Isolation

Each VM operates in:

- A secure
- Self-contained
- Independent environment

If one VM:

- Crashes
- Gets infected with malware
- Consumes excessive resources

Other VMs remain unaffected.
Isolation ensures:

Fault Containment

Two Main Types of Hypervisors

Hypervisors are classified into:

- Type 1 (Bare Metal)
- Type 2 (Hosted)

The classification depends on:

- Where the hypervisor is installed.
- How it interacts with hardware.

Type 1 Hypervisor (Bare Metal)

Location:

- Installed directly on physical hardware.
- No intermediate operating system.

Performance:

- High performance.
- Very low overhead.

Common Use:

- Enterprise data centers.
- Cloud platforms (AWS, Azure).

Examples of Type 1 Hypervisors

Common examples:

- VMware ESXi
- Microsoft Hyper-V
- KVM (Kernel-based Virtual Machine)

These are widely used in:

- Enterprise virtualization.
- Cloud infrastructure.
- Large-scale data centers.

Type 2 Hypervisor (Hosted)

Location:

- Installed as an application.
- Runs on top of an existing operating system.

Performance:

- Lower performance.
- Limited by host OS overhead.

Common Use:

- Personal computers.
- Testing and development environments.

Examples of Type 2 Hypervisors

Common examples:

- Oracle VirtualBox
- VMware Workstation

Used for:

- Running Linux on Windows.
- Running Windows on macOS.
- Development experimentation.

Comparison: Type 1 vs Type 2

Feature	Type 1	Type 2
Installation	Direct on hardware	On host OS
Performance	High	Lower
Overhead	Minimal	Higher
Use Case	Data centers	Personal/dev use
Examples	ESXi, KVM	VirtualBox

1. Consolidation

Consolidation means:

- Running multiple virtual servers on one physical machine.
- Reducing hardware requirements.
- Saving space and electricity.

Instead of:

$$1\ Server = 1\ Application$$

We achieve:

$$1\ Server = Many\ Virtual\ Servers$$

2. Snapshots and Recovery

Hypervisors allow:

- Taking a snapshot of a VM.
- Saving exact system state.
- Restoring to that exact point.

If something fails:

- Instantly revert.
- Reduce downtime.

Snapshots are powerful for:

- Testing
- Patching
- Risky updates

3. Portability

Virtual Machines are:

- Essentially files.
- Stored as disk images.
- Easily copied or moved.

This enables:

- Migration between servers.
- Backup and replication.
- Disaster recovery.

Portability increases infrastructure flexibility.

Hypervisor in the Big Picture

Hypervisor enables:

- Infrastructure virtualization.
- Cloud computing foundations.
- Multi-tenant data centers.
- Efficient hardware utilization.

It is the foundation upon which:

- Public clouds operate.
- Virtualized enterprise environments run.
- Containers often indirectly depend.

Learning Objectives

After this session, students will be able to:

- Understand Docker architecture
- Explain Docker Client and Docker Daemon
- Define Docker Images and Containers
- Differentiate Docker Hub and Docker Registry
- Describe Docker workflow in DevOps

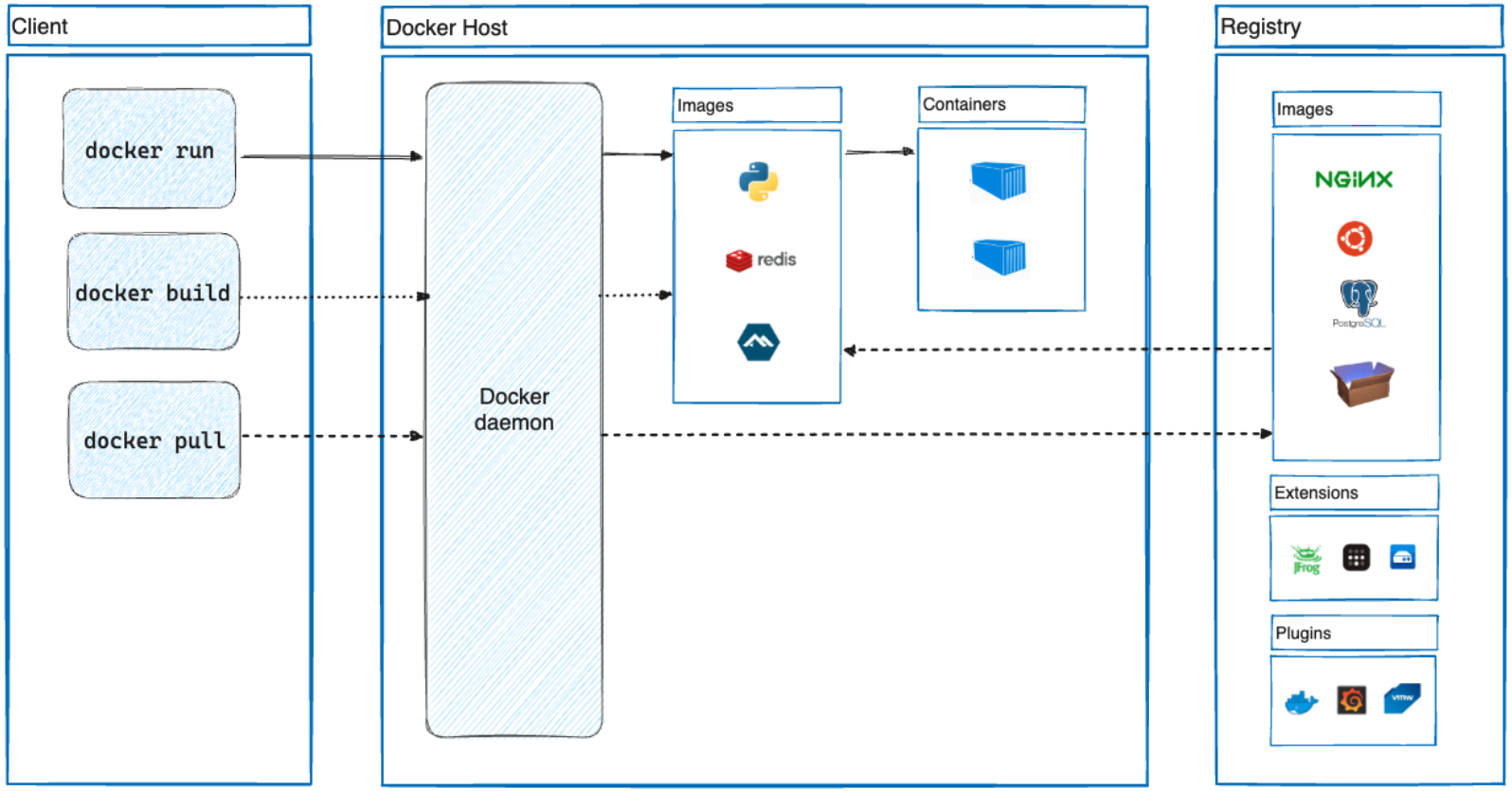
What is Docker?

Docker is a containerization platform that allows developers to:

- Package applications
- Bundle dependencies
- Ensure portability
- Deploy consistently across environments

Key Idea: Build once, run anywhere.

Docker Architecture Overview



Main Components:

- Docker Client
- Docker Daemon
- Docker Images
- Docker Containers

Docker Client

The Docker Client:

- Is the command-line interface (CLI)
- Accepts user commands
- Communicates with Docker Daemon via REST API

Examples:

- docker build
- docker run
- docker pull
- docker push

The client does NOT create containers directly — it sends requests to the daemon.

Docker Daemon (dockerd)

The Docker Daemon:

- Runs in the background
- Listens for Docker API requests
- Builds images
- Runs containers
- Manages networks and volumes

Core Role: It is the engine that performs all container operations.

Docker Images

A Docker Image:

- Is a read-only template
- Contains application code
- Includes dependencies and libraries
- Built from a Dockerfile

Images are layered:

- Base OS layer
- Application layer
- Configuration layer

Docker Containers

A Docker Container:

- Is a running instance of an image
- Is lightweight
- Shares host OS kernel
- Has isolated file system and processes

Lifecycle:

- Create
- Start
- Stop
- Remove

Docker Registry

Docker Registry:

- Is a storage and distribution system for Docker images
- Allows push and pull of images
- Can be public or private

Functions:

- Store images
- Manage versions (tags)
- Share images across teams

Docker Hub

Docker Hub:

- Is the default public Docker registry
- Hosts official images (e.g., nginx, ubuntu)
- Supports private repositories

Example:

- docker pull ubuntu
- docker push username/app:latest

Docker Hub simplifies collaboration in DevOps workflows.

Running Your First Docker Container

Basic command:

```
docker run hello-world
```

What happens internally:

- Docker client sends request to daemon
- Daemon checks if image exists locally
- If not found → pulls from Docker Hub
- Creates container from image
- Runs container
- Prints output and exits

Understanding the hello-world Image

The hello-world image:

- Smallest test image
- Verifies Docker installation
- Demonstrates image pull process

After running:

- Image is stored locally
- Container stops automatically

Listing Docker Images and Containers

List downloaded images:

```
docker images
```

List running containers:

```
docker ps
```

List all containers (including stopped):

```
docker ps -a
```

Running Containers from Versioned Images

Images have tags (versions).
Run specific version:
`docker run nginx:1.25`

If no tag specified:
`docker run nginx`

Default tag used → latest

Why Versioned Images Are Important

Benefits:

- Reproducible deployments
- Prevent breaking changes
- Stable production environments
- Better CI/CD control

Best Practice: Always specify version in production.

Running Containers in Detached Mode

Run in background:

```
docker run -d nginx:1.25
```

Detached mode:

- Container runs in background
- Terminal remains free

Check running containers:

```
docker ps
```

Stopping and Restarting Containers

Stop container:

```
docker stop <container_id>
```

Start stopped container:

```
docker start <container_id>
```

Restart container:

```
docker restart <container_id>
```

Removing Docker Containers

Remove stopped container:

```
docker rm <container_id>
```

Force remove running container:

```
docker rm -f <container_id>
```

Remove all stopped containers:

```
docker container prune
```

Docker Logging Commands

View logs of a container:

```
docker logs <container_id>
```

Follow logs in real-time:

```
docker logs -f <container_id>
```

Show last 10 lines:

```
docker logs --tail 10 <container_id>
```

Why Logging is Critical in DevOps

Logs help in:

- Debugging errors
- Monitoring application behavior
- Identifying crashes
- Performance analysis

Logs are essential in CI/CD pipelines and production monitoring.

Docker Container Lifecycle Summary

Image → Container Created → Running → Stopped → Removed

Key Commands Covered:

- docker run
- docker ps
- docker stop/start/restart
- docker rm
- docker logs

You can now:

- Run containers
- Use versioned images
- Manage lifecycle
- Inspect logs

Docker Workflow in DevOps

Typical workflow:

- Developer writes Dockerfile
- Docker Client sends build command
- Docker Daemon builds image
- Image pushed to Docker Hub/Registry
- Production server pulls image
- Container runs in cloud

Result: Fast, consistent, automated deployments.

Introduction to Docker Volumes

A Docker Volume is a mechanism for persistent data storage outside the container filesystem.

Why needed?

- Containers are ephemeral
- Data is lost when container is removed
- Applications like databases require persistent storage

Key Idea: Volumes store data independently of containers.

Why Containers Need Volumes

Problem:

- When a container stops → writable layer disappears
- Any data created inside container is lost

Solution:

- Use Docker volumes
- Store data on host machine
- Data persists even if container is deleted

Volumes are essential for stateful applications.

Creating and Managing Docker Volumes

Create a volume:

```
docker volume create myvolume
```

List volumes:

```
docker volume ls
```

Inspect volume:

```
docker volume inspect myvolume
```

Docker manages storage location automatically.

Mounting a Volume into a Container

Run container with volume:

```
docker run -v myvolume:/app/data nginx
```

What happens:

- Volume is mounted at /app/data
- Data written to this path is stored persistently
- Removing container does NOT remove volume

Volume acts as external storage for container.

Benefits of Docker Volumes

Advantages:

- Persistent storage
- Data sharing between containers
- Backup and restore capability
- Better performance than bind mounts
- Managed securely by Docker

Common Use Cases:

- Databases (MySQL, PostgreSQL)
- Logs storage
- Shared application data

Summary

- Docker Client sends commands.
- Docker Daemon executes commands.
- Images are blueprints.
- Containers are running instances.
- Registry stores images.
- Docker Hub is the public registry.